

Trained AI-Code Detectors Key on *Formatting*, Not Authorship

A semantics-preserving red-team of DroidDetect (and a null skill confound)

Autonomous experiment (Claude Code) for Matthew

May 22, 2026

Done by AI, not verified by human. This report was produced *autonomously* by an AI agent (Claude Code). Its experiments, numbers, figures, and prose have *not* been checked by a human. Treat every result and claim as provisional and independently verify before citing or relying on it.

Abstract

We set out to ask whether AI-code detectors track author **species** (human vs. LLM) or author **skill/style**, and arrived at a sharper, more cautionary answer about the one family of off-the-shelf trained code detectors we could obtain. **(1) Headline.** DroidDetect [9], a ModernBERT-based [10] detector that we load faithfully—reproducing the authors’ performance on their *own* DroidCollection Python test data (AUROC 0.999, 0.017 human false-positive rate)—has its verdict *flipped* by a *semantics-preserving* reformatting. Running its own human test code through **black** [5] (the standard formatter, which keeps comments and only restyles layout) raises the fraction it calls “machine” from **0.010** to **0.828** (`ast.unparse` gives 0.525), collapsing the human/machine separation from 0.975 to 0.149. Stripping comments *alone* barely moves it (0.030); identifiers, literals, and docstrings are preserved exactly. The operative signal is therefore layout/quote/wrap style, not authorship and not lexical content. This is consistent with a data-collection artifact (machine code is born auto-formatted, human code is scraped as-is) and a shortcut-learning failure [3], and it reproduces across *every* sibling in the family (Sec. 4.2). **(2) Consequence.** This formatting dependence explains a dramatic cross-source failure: on tidy human Codeforces Python from a *different* scrape [7] the same model labels 0.673 of ordinary human code AI, and 0.830 of contamination-free pre-LLM elite code (Torvalds’ **git**, antirez’s Redis, Hipp’s SQLite, CPython)—at roughly its own false-positive rate, i.e. a domain-shift over-flagging, not an “elite-code” effect. **(3) Skill.** On the competitive-programming difficulty axis we find a *null*: the skill confound is bounded small ($\Delta_{\text{confound}} \approx 0.003$, 95% CI upper bound 0.021 AUROC for the trained detector), with within-species slopes that do not support a skill-tracking account. **Deployment implication:** an integrity tool calibrated on one corpus can flag the majority of *legitimate* human code simply because the author ran a code formatter—a false-positive failure mode invisible to the i.i.d., fixed-threshold-free accuracy such tools report. The zero-shot statistical detectors [2, 4] do not collapse this way, but are near-random on this domain—all three detectors fail here, differently.

1 Question and hypotheses

Post-RLHF code models are tuned toward idiomatic, well-structured “senior-engineer” style. If detectors learned that style as the AI signal, then *AI-vs-human* detection partly reduces to *expert-vs-novice* detection. We pre-registered:

- **H_{species}**: detectors classify species; within-species skill has no effect (slopes ≈ 0 ; AUROC constant across skill cells).
- **H_{skill}**: detectors classify skill; among humans, higher skill $\rightarrow$ more “AI”; among LLMs, novice prompting $\rightarrow$ less “AI”.
- **H_{mixed}**: a weighted combination.

The headline finding is orthogonal to all three and emerged from a robustness check: the trained detector keys on a surface feature (formatting) that none of the hypotheses anticipated.

2 Background and related work

Zero-shot detectors score text by its own likelihood geometry under a reference LM: DetectGPT [8] via probability curvature, Fast-DetectGPT [2] via an analytic conditional-probability discrepancy, and Binoculars [4] via a cross-perplexity ratio. We use the latter two as canonical, vendored baselines. **Trained detectors** fine-tune an encoder on a labeled human/machine corpus; DroidDetect is trained on DroidCollection [9] atop ModernBERT [10] and is, to our knowledge, the strongest openly released trained *code* detector. We could not locate any independent out-of-the-box trained code detector of comparable provenance to test against; the entire family we evaluate shares these authors and this training data, which we treat as a central limitation. **Shortcut learning** [3] names the failure where a classifier exploits a spurious surface feature that is predictive in-distribution but invalid under shift. The closest prior instance for detectors is Liang et al. [6], who show that text detectors are biased against non-native English writers—a surface-feature false positive. Our result is the code analog, with layout/formatting as the shortcut. Notably the DroidDetect authors report i.i.d. F1 with a documented cross-domain collapse (their Table 3) but no fixed-FPR or calibration metrics, and their attention interpretability (their §E.2: weight on whitespace, punctuation, and blank lines) is consistent with the formatting shortcut we isolate causally here.

3 Data and methods

Human. Accepted Python-3 Codeforces solutions [7], difficulty bands easy (≤ 1200) and hard (≥ 1900). *Deviation from the original plan:* no public dataset co-locates human source code with *author* Elo, so the human skill axis is problem difficulty—a noisy, and as it turns out stylistically inverted, proxy. **LLM.** 3 models (gemma-2-2b-it, Qwen2.5-3B-Instruct, and the frontier gpt-5.4-nano¹) under natural/novice/expert prompts on the same problems. **Detectors** (oriented higher = more AI): DroidDetect-Base-Binary (trained) [9]; Fast-DetectGPT [2] and Binoculars [4] (statistical; canonical vendored implementations). **Stats.** AUROC is primary (threshold-free); all CIs use a cluster bootstrap over *problems*; the within-species score slope (z-scored detector output regressed on the skill axis) is the mechanism test; Δ_{confound} is reported per detector. We apply no multiple-comparisons correction across the (detector $\times$ species $\times$ band) intervals and accordingly do not lean on any single CI’s exclusion of zero.

¹gpt-5.4-nano is OpenAI’s nano-tier model of the GPT-5.4 generation, accessed via the OpenAI API in May 2026.

4 Results

4.1 DroidDetect’s verdict is gated by formatting (headline)

We first establish the detector is loaded correctly: on the authors’ own DroidCollection Python test set it scores AUROC 0.999 with a 0.017 human false-positive rate and 0.983 machine recall—reproducing the published numbers. So the behavior below is a property of the model, not a loading bug. We then apply *semantics-preserving* reformatters to the input code and re-score, using two deliberately: (i) **black**, which restyles whitespace/indentation/quote-style/ line-wrapping but *keeps comments and docstrings*; (ii) `ast.unparse(ast.parse(code))`, which additionally drops `#` comments. Neither alters the lexical content a detector might legitimately use—variable and function names, string/numeric literals, and docstrings are all retained (we verify the identifier-token multiset is preserved at a mean of 1.000, i.e. exactly). These are *formatting* changes, not “humans and AI use different words” changes.

The effect is dramatic (Fig. 1). On its *own* human test data the fraction called “machine” rises from 0.010 (raw) to 0.828 under **black** (comments *kept*) and 0.525 under `ast.unparse`; the human↔machine gap collapses from 0.975 to 0.149. That **black** alone—which changes layout, quote-style, and line-wrapping but not words or comments—produces nearly the whole flip, while *removing comments* on their own barely moves it (0.030), pins the operative signal to **layout/quote/wrap style**. Two further observations sharpen the causal reading. First, the mean predicted $P(\text{machine})$ on this human corpus moves from near zero to near one—this is a full distributional shift, not a knife-edge of borderline cases crossing the decision boundary. Second, the effect is *asymmetric*: reformatting already-clean DroidCollection-*machine* code is a near-identity (it stays ≈ 0.977 “machine”), exactly as expected if the learned rule is “clean layout = machine.” The statistical detectors shift far less. We therefore describe DroidDetect’s decision as *gated by formatting*: it separates human from machine well *within* a fixed formatting regime, but a formatter moves human code across the boundary.

Why—a training-data formatting confound (suggestive). This shortcut is surprising, since humans run **black** too. The likely origin is in the data: the two training classes differ in formatting cleanliness. As a coarse proxy, a fraction 0.149 of *human* DroidCollection Python is already **black**-compliant versus 0.443 of *machine* Python (mean similarity-to-**black** 0.837 vs 0.946)—a roughly threefold gap. Machine code is born clean—freshly generated by LLMs that emit canonical PEP8-ish layout—while human code carries its original formatting, so a loss-minimizing classifier can exploit this almost-free cue instead of learning authorship. We flag this metric as *suggestive rather than decisive*: a string-similarity-to-**black** ratio is sensitive to code length and content, and we did not length-control it, so the magnitude of the 15-vs-44% gap should not be over-read. The causal claim of the headline rests on the *within-corpus* ablation (Fig. 1), where source is held fixed; the data statistic only proposes a mechanism. Because real human code is routinely auto-formatted, the cue is invalid at deployment regardless of its exact training-set magnitude.

4.2 The confound is family-wide, not a single checkpoint

A web audit found no clean independent out-of-the-box trained *code* detector to compare against, so we tested all 3 published DroidDetect siblings—different sizes (ModernBERT base/large) and granularities (2- and 4-class)—through the same probe (Table 1). The black-flip reproduces on every one: in-distribution human code goes from at most 0.027 “machine” to between 0.764 and 0.831. Scaling to the Large encoder does *not* fix it. This rules out “a quirk of one checkpoint” and

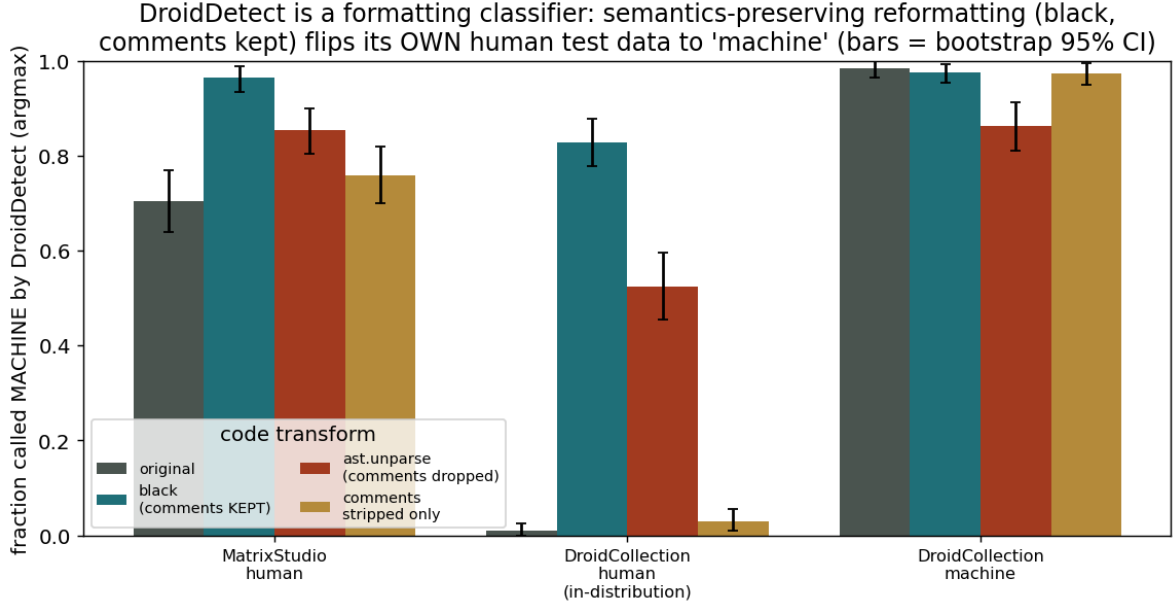


Figure 1: Semantics-preserving reformatting flips DroidDetect’s verdict on the authors’ *own* human test data. Running it through **black** (comments kept) raises the “machine” rate from 0.010 to 0.828 (**ast.unparse**: 0.525); comment-stripping alone barely moves it (0.030); already auto-formatted *machine* code is essentially unchanged. Identifiers and literals are preserved exactly, so the operative signal is layout/quote/wrap style.

“the small model is undertrained,” and is consistent with a cause in the shared training data. It does *not*, by itself, prove the cause is the data rather than the shared recipe, because all siblings share authors, architecture family, and DroidCollection; the decisive test—an independent architecture trained on the same data—is left to future work (Sec. 5).

Detector	classes	DC-human orig.	DC-human black	DC-machine	MS-human	legendary
Base-Binary	2	0.013	0.831	0.987	0.713	0.830
Base	4	0.027	0.764	1.000	0.780	0.911
Large-Binary	2	0.007	0.811	0.980	0.700	0.763

Table 1: Fraction called “machine” (native argmax) across the DroidDetect family. Every sibling flips its own in-distribution human code from near zero to a large majority under **black** (bold), over-flags cross-source human code (MS), and flags provably-human pre-LLM elite code (legendary). Scaling to Large does not help.

4.3 The skill confound is a (bounded) null on the competitive-skill axis

We find no evidence that competitive-programming skill confounds detection—but this is a *null*, and we are explicit about its limits. The skill-matched vs. mismatched AUROC gap is small for all detectors (Table 3, Fig. 3): $\Delta_{\text{confound}} = 0.011$ (CI [0.001, 0.021]) for DroidDetect, -0.004 and 0.002 for the statistical detectors. The CI bounds the confound below ≈ 0.021 AUROC—we can rule out a *large* skill confound on this axis, not assert a true zero. Two caveats temper even this: (i)

the matched/mismatched AUROCs themselves are mediocre (0.746 / 0.757 for DroidDetect; 0.704 / 0.665 for the statistical detectors), so $\Delta \approx 0$ partly reflects a *floor*—there is little performance left to collapse on terse competitive code; (ii) human code is accepted-only, a selection collider (rating $\rightarrow$ accept $\leftarrow$ difficulty).

Within-species slopes (Table 2, Fig. 5) point, if anything, *against* a skill-tracking account: the human slope is negative for DroidDetect (-0.30 , CI $[-0.53, -0.07]$) and Fast-DetectGPT (-0.30), and null for Binoculars (0.07). We do *not* interpret the negative sign as “skill inverts”: the accepted-only collider can manufacture a negative slope by selecting atypical (terse, hyper-optimized) hard-problem solutions. The defensible reading is only that higher competitive-programming difficulty skill is *not* scored more AI-like, contrary to H_{skill} .

Detector	Family	Human slope (hard $\rightarrow$ easy)	LLM slope (novice $\rightarrow$ expert)
DroidDetect	trained	-0.30	-0.14
Fast-DetectGPT	statistical	-0.30	-0.54
Binoculars	statistical	0.07	-0.19

Table 2: Within-species skill slopes (z-scored detector score). Positive = skill-tracking.

Detector	Family	AUROC matched	AUROC mismatched	Δ_{confound}
DroidDetect	trained	0.746	0.757	0.011
Fast-DetectGPT	statistical	0.704	0.700	-0.004
Binoculars	statistical	0.665	0.667	0.002

Table 3: AUROC by skill cell and Δ_{confound} (cluster-bootstrap over problems).

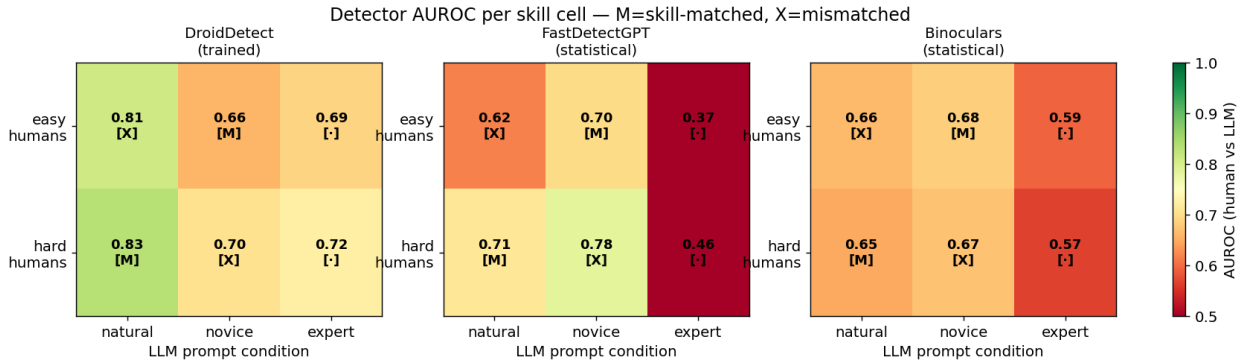


Figure 2: AUROC per (detector, difficulty band, prompt condition). M = skill-matched, X = mismatched.

4.4 Cross-source generalization failure (the dominant effect)

DroidDetect is a softmax classifier whose actual prediction is the *argmax* over {human, machine}, with no tunable threshold; we report it at that native operating point and measure its *true* false-positive rate on ordinary human code. On clean human Codeforces Python from a *different* source [7] the FPR explodes from 0.017 (in-distribution) to 0.673: that fraction of *ordinary*

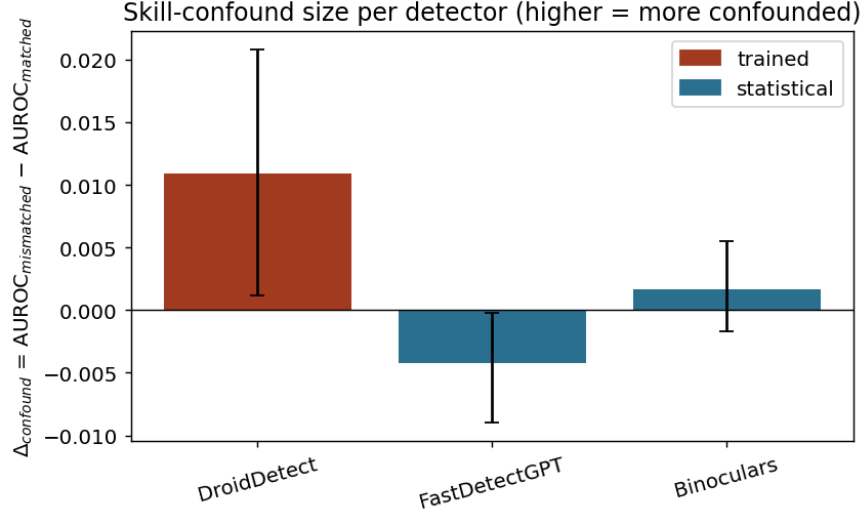


Figure 3: Δ_{confound} per detector with cluster-bootstrap 95% CIs; all intervals sit near zero, bounding any competitive-skill confound to be small on this axis.

human solutions are called AI (vs. 0.843 of actual Codeforces LLM code—so it barely separates the two out of distribution). On the provably-human pre-LLM elite corpus the rate is 0.830—essentially the *same* as its cross-source human FPR (0.673). We therefore read the legendary result as the *same* domain-shift/formatting over-flagging affecting all human code here, **not** a distinct “elite code looks AI” effect. Per-author rates do vary (CPython 0.975, Hipp 0.825, antirez 0.775, Torvalds’ original git 0.600), but with as few as 15 samples per author and Torvalds sitting *below* the ordinary-human baseline, we do not interpret an ordering. The statistical detectors, shown at a fixed 5%-FPR operating point for comparability, flag the same code far less (Fast-DetectGPT 0.126, Binoculars 0.200)—but recall they are near-random as detectors on this domain, so this is robustness, not usefulness.

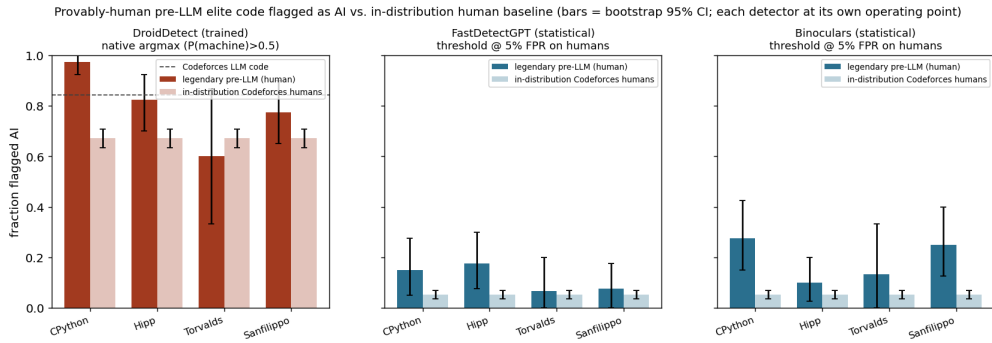


Figure 4: Provably-human pre-LLM elite code flagged as AI (solid), with the in-distribution ordinary-human baseline beside each (semitransparent); bootstrap 95% CIs. DroidDetect is shown at its native argmax operating point (baseline = its true cross-source human FPR, 0.673); the statistical detectors at a fixed 5%-FPR point.

4.5 Mechanism and confirmatory checks

Surface style features explain $R^2 = 0.108$ of DroidDetect’s score and 0.276 of Fast-DetectGPT’s—style matters but does not fully account for the trained detector. On the harder Codeforces problems human code is in fact *more* idiomatic (type-hint rate $0.594 \rightarrow 0.755$, Fig. 6), which makes the negative human slope notable: more idiomatic *competitive* code is still not read as more AI.

A cohort using real *author* Elo (AtCoder [1]; low band $n = 45$ mean Elo 323, high band $n = 9$ mean Elo 2225) is directionally consistent—the high–low score gap is negative for all detectors (e.g. DroidDetect -0.42 SD, Fig. 7)—but the high band is far too thin ($n = 9$) for its slope CI to exclude zero, so we treat it as suggestive only, not as confirmation.

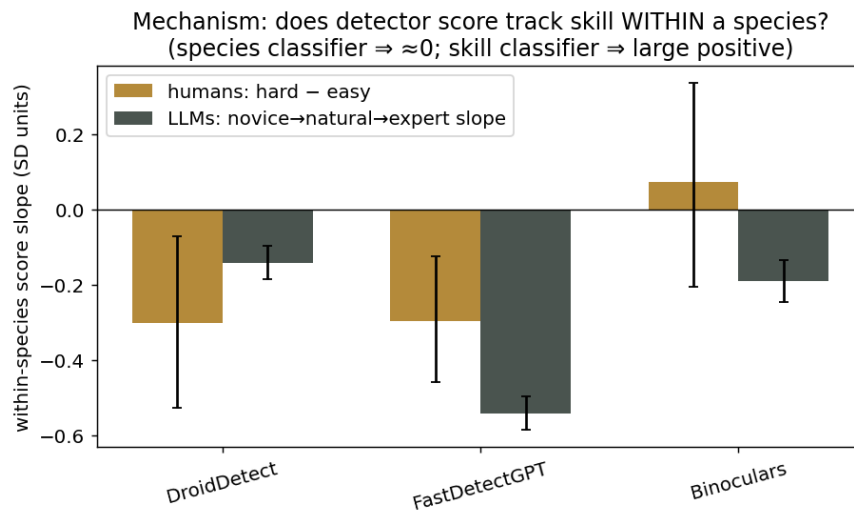


Figure 5: Within-species slopes: detector score vs. skill, per species.

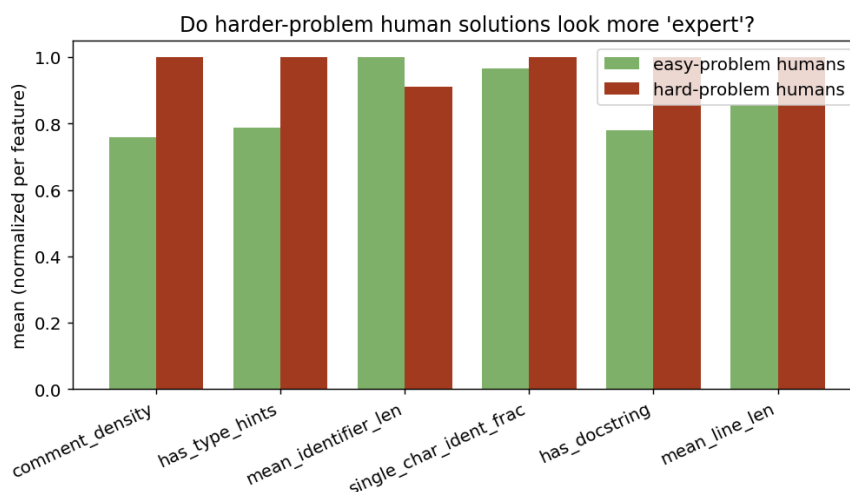


Figure 6: Human code style vs. problem difficulty: harder problems draw *more* idiomatic code (higher type-hint rate), so the difficulty proxy is stylistically inverted relative to the “senior-engineer” register that motivates H_{skill} .

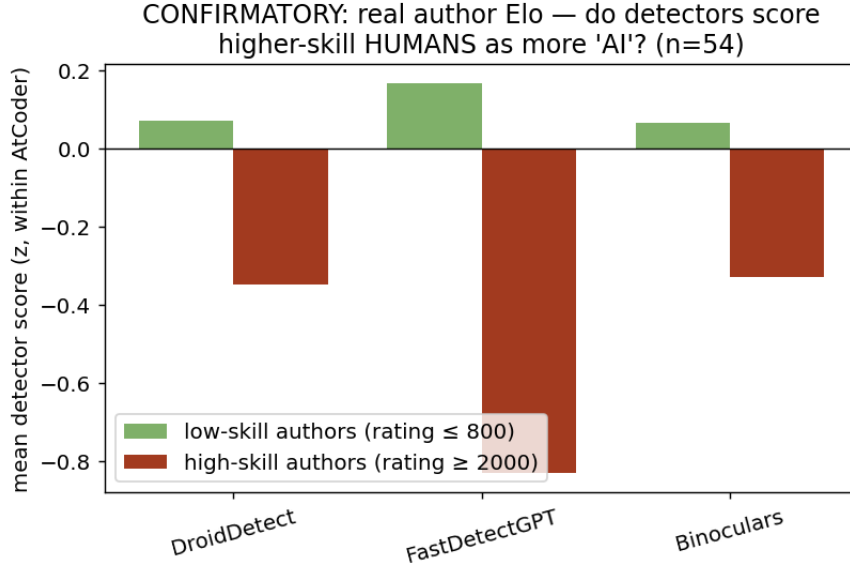


Figure 7: AtCoder author-Elo cohort: detector score vs. real author rating. Directionally consistent with the difficulty-axis null but underpowered (high band $n = 9$).

5 Limitations

- **Single-corpus red-team.** Every trained detector here is the DroidDetect family: same authors, same DroidCollection training data, same ModernBERT backbone. We show the formatting shortcut is family-wide and not a single-checkpoint quirk, but whether it generalizes beyond this data *recipe* is untested. The decisive experiment—training an *independent* architecture (e.g. CodeBERT/Roberta) on DroidCollection and re-running the ablation—would separate “the data is confounded” from “the DroidDetect recipe is confounded” and also rule out a ModernBERT-tokenizer artifact; it is proposed, not run.
- **Formatting metric is suggestive.** The human-vs-machine black-compliance gap (0.149 vs 0.443) uses an un-length-controlled string-similarity ratio; it motivates the mechanism but is not the load-bearing evidence (the within-corpus ablation is).
- **Problem difficulty / rating $\neq$ idiomatic-code skill**, and the proxy is stylistically inverted; the skill result is a bounded null, not a true zero.
- **Accepted-only human code** is a selection collider; LLM code is unfiltered—asymmetric.
- **Small LLM panel** (gemma-2-2b, Qwen2.5-3B, gpt-5.4-nano) and **thin AtCoder high-skill band** ($n = 9$); preliminary n throughout.
- **MatrixStudio is a different human source** than DroidCollection’s training data; we cannot fully separate “source shift” from “formatting” for the cross-source FPR, though the within-corpus ablation isolates formatting and the pre-LLM legendary probe is the contamination-free anchor.

6 Reproducibility

All numbers in this report are emitted by scripts into `paper/macros.tex` and referenced by name; all bibliography metadata is fetched from arXiv into `paper/refs.bib`. The full pipeline reproduces end-to-end via `./run_pipeline.sh` (single GPU). Detector checkpoints are the released `project-droid/DroidDetect-*` weights, loaded verbatim from their model card; the zero-shot detectors use vendored upstream implementations. Generation uses HuggingFace `transformers` (the local vLLM build is ABI-incompatible), and `black` is invoked through `uvx` so the shared environment is untouched. Seeds and model revisions are recorded in the repository.

7 Conclusion

The headline is mechanistic and cautionary: the strongest off-the-shelf trained code detector we could obtain has its decision **gated by formatting**. Faithfully loaded (AUROC 0.999, 0.017 FPR on the authors’ own data), it nonetheless flips from 0.010 to 0.828 “machine” on that *same* human data under a semantics-preserving reformatting, and the flip reproduces across every sibling in the family. That dependence explains the rest: 0.673 of ordinary human Codeforces code from a different scrape, and 0.830 of provably-human pre-LLM elite code, are flagged not because they are AI-like or expert but because their formatting differs from the training-human formatting. Secondarily, “does detection reduce to *skill*?”—not to competitive-programming skill, where the confound is a bounded null. The zero-shot statistical detectors do not collapse under formatting, but are near-random here, so no detector we tested is deployable on this domain; they fail differently. The deployment implication is stark and is a *false-positive* risk, not an evasion trick: a student or developer whose only “offense” is running an autoformatter can be flagged as AI by an integrity tool of this kind, and the tool’s reported accuracy—all i.i.d., with no fixed-FPR or calibration reporting—hides this. Clean follow-ups: an independent-architecture replication of the ablation, evaluation under format-normalization and across held-out human *sources*, and calibrated fixed-FPR operating points.

References

- [1] AtCoder Inc. AtCoder programming contests. <https://atcoder.jp>, 2024.
- [2] Guangsheng Bao, Yanbin Zhao, Zhiyang Teng, Linyi Yang, and Yue Zhang. Fast-detectgpt: Efficient zero-shot detection of machine-generated text via conditional probability curvature, 2023. URL <http://arxiv.org/abs/2310.05130v3>.
- [3] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks, 2020. URL <http://arxiv.org/abs/2004.07780v5>.
- [4] Abhimanyu Hans, Avi Schwarzschild, Valeriia Cherepanova, Hamid Kazemi, Aniruddha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Spotting llms with binoculars: Zero-shot detection of machine-generated text, 2024. URL <http://arxiv.org/abs/2401.12070v3>.
- [5] Łukasz Langa and Black contributors. Black: The uncompromising Python code formatter. <https://github.com/psf/black>, 2020.

- [6] Weixin Liang, Mert Yuksekgonul, Yining Mao, Eric Wu, and James Zou. Gpt detectors are biased against non-native english writers, 2023. URL <http://arxiv.org/abs/2304.02819v3>.
- [7] MatrixStudio. Codeforces-Python-submissions. <https://huggingface.co/datasets/MatrixStudio/Codeforces-Python-Submissions>, 2024.
- [8] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. Detectgpt: Zero-shot machine-generated text detection using probability curvature, 2023. URL <http://arxiv.org/abs/2301.11305v2>.
- [9] Daniil Orel, Indraneil Paul, Iryna Gurevych, and Preslav Nakov. Droid: A resource suite for ai-generated code detection, 2025. URL <http://arxiv.org/abs/2507.10583v3>.
- [10] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference, 2024. URL <http://arxiv.org/abs/2412.13663v2>.